RAGS: Reinforcement Learning-based Global Scheduling for LLM Agent Serving Systems

Anonymous Author(s)

Abstract

LLM-based Agent systems compose multiple inference calls into dynamic execution graphs, posing three fundamental challenges for global scheduling: nonlinear topologies with conditional branches and feedback loops, heterogeneous per-node inference costs, and KV cache affinity in tension with load balance. Existing schedulers address these only partially: topology-aware queuing breaks down when dynamic branches invalidate static stage estimates, while Round-Robin and Session Affinity represent opposite extremes that trade off cache reuse against load balance without adaptation. We present **RAGS** (RL-based Agent Global Scheduler), a two-tier framework that co-designs queuing and dispatch to address these challenges jointly. The queuing tier, **DynA**, prioritizes requests via an online-estimated conditional near-completion probability over per-Agent-type completion-round distributions, augmented with cumulative-latency-aware ordering and anti-starvation, all without relying on any predefined topology. The dispatch tier models request-to-instance assignment as an MDP and trains a Transformer-based policy with Double-DQN to jointly optimize KV cache reuse, load balance, and queue congestion via offline pretraining and online fine-tuning. Experiments on a simulated 4-instance LLM cluster across three representative Agent workloads show that RAGS reduces mean end-to-end latency by up to **5.2**× over Round-Robin and **3–4**× over Parrot and Ayo; ablation and heterogeneous GPU cluster experiments further confirm the complementary contributions and generalizability of both tiers.

CCS Concepts

• **Do Not Use This Code → Generate the Correct Terms for Your Paper**; *Generate the Correct Terms for Your Paper*; Generate the Correct Terms for Your Paper; Generate the Correct Terms for Your Paper.

Keywords

LLM serving, Agent workflows, global scheduling, reinforcement learning, KV cache, load balancing, queuing algorithms

ACM Reference Format:
Anonymous Author(s). 2018. RAGS: Reinforcement Learning-based Global Scheduling for LLM Agent Serving Systems. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX).* ACM, New York, NY, USA, 10 pages. https://doi.org/XXXXXXX.XXXXXXX

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, Woodstock, NY
© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Large Language Models (LLMs) have rapidly evolved from standalone research artifacts into the computational backbone of diverse intelligent applications. Modern LLM-based Agent systems (e.g., multi-turn chatbots, autonomous code generation pipelines, and large-scale document processing workflows) inherently orchestrate multiple inference calls into structured, multi-step execution graphs. As these deployments scale from research prototypes to production services handling thousands of concurrent workflows, the efficiency of the underlying inference infrastructure has emerged as a critical bottleneck. In this context, the global scheduler within a multi-tenant LLM serving cluster assumes a critical role: it must simultaneously dictate the execution priority of pending requests and route them to optimal backend instances, with the primary objective of minimizing end-to-end workflow latency under highly dynamic and heterogeneous workloads.

Classical scheduling strategies designed for stateless, single-turn inference fall short in these dynamic scenarios, as they rely on assumptions that Agent workloads inherently contradict. First, Agent workflows feature dynamic, nonlinear topologies, such as conditional branches and iterative repair loops. Because actual execution paths and depths are resolved only at runtime, static graph-based priority estimation becomes ineffective. Second, requests generated across different stages of a single workflow are heterogeneous in input/output lengths and GPU memory footprints. Consequently, uniform scheduling treatment typically induces head-of-line blocking and resource imbalance. Third, adjacent calls within a workflow contain overlapping context prefixes. This presents KV cache reuse opportunities, forcing a strict trade-off between cache locality and cluster-wide load balancing.

Existing approaches only partially address these scheduling challenges. On the queuing side, FCFS remains unaware of workflow progress, propagating queuing delays along dataflow edges. Topology-aware strategies like Ayo [22] prioritize requests based on static residual stage counts. However, they become ineffective when dynamic branches and loops cause the estimated stage depth to deviate from the actual execution trajectory. On the routing front, Round-Robin policies achieve load balance at the expense of prefix-cache locality. Conversely, Session Affinity, as adopted by end-to-end systems such as Parrot [14], maximizes cache reuse via static session binding. Yet, this approach induces hardware hotspots when parallel fan-out stages concentrate requests on a single instance, and it lacks the adaptability to handle dynamic load shifts, frequently resulting in cluster-wide load imbalances. Taken together, these limitations indicate that effective global scheduling for LLM Agent serving necessitates the co-design of queuing and

routing algorithms, as neither tier can achieve robust performance in isolation.

In response to these challenges, we present **RAGS** (RL-based Agent Global Scheduler), a co-designed global scheduling framework that addresses both queuing and routing dimensions in a coordinated manner. The queuing layer, **DynA**, determines request priorities using exclusively runtime-observable signals (specifically, online-estimated completion-round distributions, accumulated workflow latencies, and per-request queue ages), operating independently of any predefined workflow topology. Concurrently, the routing layer employs a Reinforcement Learning (RL) policy that models request-to-instance assignment as a Markov Decision Process. By training a Transformer-based network via Double-DQN, it jointly optimizes KV cache reuse, cluster load balance, and queue congestion, ensuring continuous adaptability through offline pre-training coupled with online fine-tuning.

The main contributions of this paper are as follows.

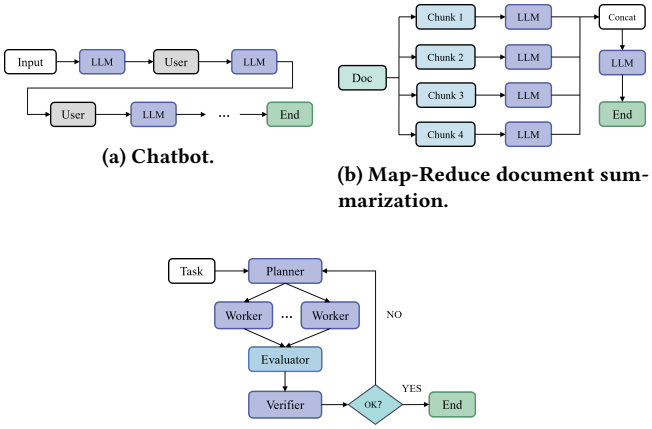
- We propose an adaptive queuing algorithm that estimates the overall completion probability of active workflows using statistics gathered at runtime. By prioritizing individual requests based on the execution progress of their parent workflows, this approach effectively mitigates head-of-line blocking among concurrent tasks.
- To the best of our knowledge, RAGS is the first to apply reinforcement learning to global routing in LLM Agent serving. By jointly encoding heterogeneous cluster features, including KV cache states, load distributions, and GPU hardware configurations, the RL policy realizes an adaptive, multi-objective routing strategy that strictly outperforms traditional static heuristics.
- Evaluations on a simulated LLM serving cluster demonstrate that RAGS reduces mean end-to-end workflow latency by up to **5.2**× compared to a normal Round-Robin baseline, and by **3–4**× against Parrot and Ayo. Furthermore, ablation studies and experiments on heterogeneous GPU clusters validate the individual effectiveness and hardware adaptability of the co-designed queuing and routing modules.

2 Background and Problem Analysis

Conventionally, LLM serving systems employ a layered scheduling architecture in which the global scheduler manages two interconnected operations: (1) determining the execution priority of pending requests, and (2) routing each request to an appropriate backend instance. This section characterizes the distinctive structural properties of LLM Agent workloads to demonstrate why conventional request-level heuristics prove inadequate for these tasks. By exposing these systemic limitations, we motivate the necessity of the comprehensive global scheduling framework detailed in Section 3, which systematically addresses both the queuing and routing dimensions.

2.1 Characteristics of LLM Agent Workflows

To systematically characterize LLM Agent workloads, we surveyed mainstream open-source projects on GitHub [3, 9, 17] and analyzed representative paradigms, including chatbots, code generation, and



(a) Chatbot.

(b) Map-Reduce document summarization.

(c) Code generation.

Figure 1: Representative LLM Agent workflow structures.

document summarization. As illustrated in Figure 1, a typical Agent workflow can be abstracted as a directed graph. In this structural model, each node represents an individual LLM inference call or tool invocation, while each edge encodes a specific dataflow or control dependency. Based on this analysis, we identify three distinctive system-level traits inherent to Agent workloads.

Dynamic, nonlinear topologies with mixed concurrency. Agent workflows incorporate conditional branches, feedback loops, and interleaved sequential and parallel stages (e.g., iterative repair cycles and Map-phase fan-outs). Consequently, these features result in dynamically changing execution paths, rendering static graph-based scheduling approaches ineffective.

Heterogeneous inference costs across workflow nodes. Nodes serving different roles (such as routing, generation, and quality review) emit requests with varying prompt lengths, output lengths, and execution latencies. Because these resource requirements can differ by over 25× within a single application [6], scheduling decisions must be strictly cost-aware.

Deep context sharing and KV cache affinity. Adjacent calls within the same workflow contain overlapping context prefixes, presenting KV cache reuse opportunities. However, affinity-driven routing inherently concentrates requests and induces node-level hotspots, forcing a strict trade-off between cache locality and cluster-wide load balancing.

Taken together, these architectural traits inherently contradict the classical assumption of independent, homogeneous requests. The following two subsections examine the concrete limitations that arise when standard queuing and routing policies are applied to Agent workloads.

2.2 Limitations of Existing Queuing Algorithms

Conventional LLM serving paradigms model requests as independent, single-turn calls, where FCFS queuing is sufficient for fairness. Agent workloads, however, comprise interdependent requests that introduce three fundamental scheduling challenges.

(1) Head-of-line blocking from workflow progress heterogeneity. Concurrent Agent requests exhibit varying execution

progress, meaning the system simultaneously processes initial LLM calls and near-terminal tasks. Because FCFS is progress-agnostic, it frequently stalls near-completion requests behind newly admitted early-stage tasks, directly inflating end-to-end workflow latencies.

(2) Cascading amplification and accumulation of queuing delays. Queuing delays experienced at any individual node propagate along dataflow dependencies. Furthermore, because FCFS ranks requests solely by arrival time, it ignores accumulated workflow-level latencies. Consequently, tail performance degrades as delays compound across successive stages.

(3) Inability to handle dynamic workflow topologies. Topology-aware strategies like Ayo [22] prioritize tasks with fewer remaining stages based on a static workflow graph. However, when dynamic branches or loops execute at runtime, the statically estimated residual stage counts deviate from actual execution paths, causing tasks within active loops to be improperly under-prioritized.

Collectively, these limitations necessitate an adaptive queuing policy that tracks actual workflow progress and accumulated latencies independent of static topologies.

2.3 Limitations of Existing Routing Strategies

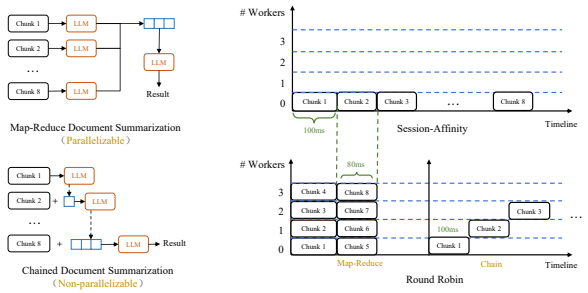


Figure 2: RR versus Session Affinity under parallel and sequential document-summarization Agent workflows.

Conventional global routing mechanisms for LLM serving typically utilize Round-Robin (RR) to distribute requests sequentially for load balancing, or Session Affinity (SA) to bind session requests to a fixed instance for maximizing KV cache reuse. However, both strategies exhibit structural limitations when processing Agent workloads.

RR ignores request heterogeneity. Because requests from distinct functional nodes exhibit diverse context lengths and GPU memory footprints, RR's uniform routing often concentrates memory-intensive tasks on a single instance. This concentration saturates GPU memory and triggers request preemptions. Consequently, preempted tasks incur re-queuing and recomputation overheads, directly degrading overall system throughput.

SA mishandles intra-workflow concurrency. During parallel execution stages (e.g., the Map phase in document summarization), workflows emit numerous concurrent subtask requests. By binding all these requests to a single instance, SA creates local hardware hotspots while leaving other instances underutilized, thereby compromising cluster-wide load balancing.

As illustrated in Figure 2, RR achieves lower latency on parallel workflows by exploiting cluster-wide concurrency, whereas SA outperforms RR on sequential workflows by preserving KV cache locality across stages. Because neither policy dominates universally, the optimal routing choice depends jointly on execution topologies, temporal dependencies, and real-time cache states.

To address these limitations, recent systems incorporate richer heuristics, such as multi-objective linear weighting [21]. However, LLM cluster states evolve continuously: request arrival patterns, instance queue lengths, and memory utilization fluctuate rapidly. Furthermore, each routing decision reshapes the environment encountered by subsequent requests. Static, fixed-weight heuristics lack the agility to capture these complex state transitions or to optimize the trade-off between immediate scheduling gains and long-horizon throughput.

Collectively, the isolated failures of existing queuing and routing algorithms necessitate the holistic global scheduling framework detailed in Section 3.

3 Algorithm Design

3.1 System Architecture

Figure 3 illustrates the overall system architecture and request processing pipeline of RAGS. The system frontend accepts heterogeneous Agent workflows and forwards all LLM inference requests to the global scheduler for centralized management. Within the scheduler, requests first enter the global queue, where the adaptive queuing module, **DynA**, calculates priorities and reorders pending tasks. Subsequently, the RL-driven routing policy asynchronously assigns these requests to target instances by evaluating queue characteristics alongside cluster states. Following inference completion, the system triggers subsequent workflow stages. Concurrently, backend instances continuously report load metrics back to the scheduler to maintain an accurate environmental state. Once the workflow concludes, the system returns the final result to the user.

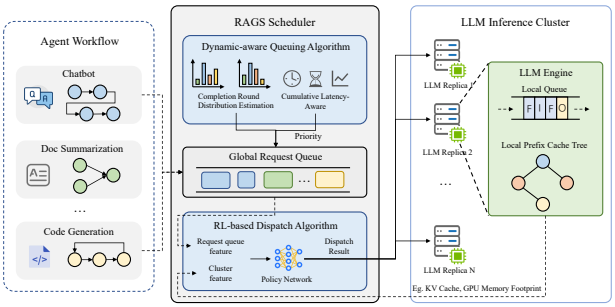


Figure 3: RAGS system architecture.

Building upon this architectural overview, the following subsections detail the two core scheduling components: the queuing algorithm and the routing policy.

3.2 DynA: Dynamic-Aware Queuing Algorithm

Due to the dynamic branching and autoregressive uncertainty inherent in Agent workflows, traditional queuing policies that rely on

static topologies prove inadequate. These methods assume a priori knowledge of the workflow structure, a condition rarely satisfied in actual deployments.

To overcome this reliance, we introduce **DynA** (Dynamic-Aware Queuing). The core principle of DynA is to derive all priority decisions exclusively from runtime-observable signals, including online-estimated completion-round distributions, accumulated workflow latencies, and per-request queue ages. By doing so, it operates entirely independently of predefined workflow topologies. Specifically, the algorithm functions across two dimensions: inter-Agent priority ordering governed by actual workflow progress, and intra-Agent request ordering driven by accumulated latencies.

3.2.1 Workflow-Progress-Aware Inter-Agent Priority. To differentiate the execution progress of requests from different Agent types (and thus prioritize requests in the later stages of their workflows) without knowing the workflow topology, DynA introduces an *online completion round distribution* mechanism.

Let $F_a(n)$ denote the empirical CDF of completion rounds for Agent type a, i.e., the probability that the total number of LLM calls in any completed workflow instance of this type does not exceed n. This distribution is maintained at runtime by continuously collecting statistics from completed workflows, without any prior assumption about static workflow topology.

Specifically, each time an Agent workflow of type a completes, the system records the completion round count n_j and updates $F_a(n)$ via Exponential Moving Average (EMA):

$$F_a^{(t+1)}(n) = (1-\lambda)\cdot F_a^{(t)}(n) + \lambda \cdot \mathbb{1}[n_j \le n], \tag{1}$$

where $\lambda \in (0,1)$ is the smoothing coefficient and $\mathbb{1}[\cdot]$ is the indicator function. By assigning higher weight to recent observations, EMA enables $F_a(n)$ to adapt to dynamic workload changes while filtering out stale historical data.

For Agent types with no historical records, the system initializes F_a with a uniform prior on $[1, N_{\max}]$:

$$F_a(n) = \begin{cases} 0, & n < 1 \\ n/N_{\max}, & 1 \le n \le N_{\max} \\ 1, & n > N_{\max} \end{cases}.$$

During online operation, EMA drives this prior toward the true empirical distribution.

Based on this real-time empirical distribution, when the scheduler receives a request from Agent type a that has already completed k LLM calls, it computes the *conditional near-completion probability* $\hat{P}_{a,k}$, defined as the probability of reaching the terminal state within the next Δ calls given that k calls have already been made:

$$\hat{P}_{a,k} = P(k \le N_a \le k{+}\Delta \mid N_a \ge k) = \frac{F_a(k{+}\Delta) - F_a(k{-}1)}{1 - F_a(k{-}1)}, \tag{2}$$

where N_a is the random variable representing total LLM calls for Agent type a, and Δ is a look-ahead window size (default Δ=1). A larger $\hat{P}_{a,k}$ indicates a higher probability of near-term completion, so DynA assigns higher scheduling priority to requests with larger $\hat{P}_{a,k}$. This design borrows from the Shortest Job First (SJF) principle while avoiding any dependence on a priori topology.

3.2.2 Cumulative-Latency-Aware Intra-Agent Ordering. For concurrent requests of the same type and same round, DynA applies an intra-Agent ordering strategy based on workflow start time. Let t_r^{start} be the initial trigger time of the workflow containing request r; the system prioritizes requests with earlier t_r^{start}, compensating requests that have already accumulated longer delays.

To prevent starvation of long-horizon Agent types under this priority ordering, DynA introduces a queue-age-aware anti-starvation mechanism. Define the current queuing wait time of request r as $\tau_r = t_{\text{current}} - t_r^{\text{arrive}}$. When τ_r reaches a system-defined maximum tolerance threshold $\tau_{\max}$, the following priority boost is triggered:

$$\text{Boost}(r) = \max\left(0,\ \frac{\tau_r - \tau_{\max}}{\tau_{\max}}\right). \tag{3}$$

Combining the conditional near-completion probability with the anti-starvation mechanism, DynA computes a comprehensive priority score for each queued request r:

$$\text{Score}(r) = \hat{P}_{a_r,k_r} + \eta \cdot \text{Boost}(r), \tag{4}$$

where a_r is the Agent type of r, k_r is the total LLM call count so far for the workflow instance, and η is the anti-starvation weight hyperparameter. Requests with higher scores are scheduled first; ties are broken by Agent start time. The complete DynA policy is given in Algorithm 1.

Algorithm 1 Dynamic-Aware Queuing Algorithm (DynA)

Require: Request queue Q; online completion-round distributions $\{F_a\}$; anti-starvation threshold $\tau_{\max}$; weight η
Ensure: Next request to dispatch r^*

```
for each request r ∈ Q do
    Retrieve Agent type a_r and completed call count k_r
    if F_{a_r} is not yet initialized then
        F_{a_r}(n) ← min(n, N_max)/N_max    // uniform prior
    end if
    P̂_{a_r,k_r} ← (F_{a_r}(k_r+Δ) − F_{a_r}(k_r−1)) / (1 − F_{a_r}(k_r−1))
    τ_r ← t_current − t_r^arrive
    Boost(r) ← max(0, (τ_r − τ_max)/τ_max)
    Score(r) ← P̂_{a_r,k_r} + η · Boost(r)
end for
Sort all requests in Q by (Score(r) desc, t_r^start asc)
r* ← first request in Q
return r*
```

3.3 RL-based Dispatch Algorithm

3.3.1 Problem Formulation and MDP Definition. We formalize the global scheduler's dispatch process as a Markov Decision Process (MDP) and solve for the optimal dispatch policy using reinforcement learning.

Consider a service system with K homogeneous LLM instances, each equipped with GPU G and deploying model M. During system operation, N Agent workflows arrive online over time. At any decision step t, let Q_t denote the set of pending requests in the global scheduler. The scheduler assigns a target LLM instance a_t to each request $q_{i:k}$ (the k-th call of workflow w_i). The optimization

objective is to minimize the average end-to-end latency across all N workflows:

$$\min_{a_t} \frac{1}{N} \sum_{i=1}^{N} l_i, \tag{5}$$

where l_i is the actual end-to-end completion time of w_i.

Since Q_t may contain multiple concurrent requests, a centralized joint assignment would require an action space of $O(|Q_t| \times K)$. To avoid the curse of dimensionality, we adopt a progressive decision paradigm: the agent assigns one request at a time to a target instance.

The MDP is defined as follows. Every Δt, the global scheduler determines target LLM instances for all requests in the waiting queue. At step t, the agent observes state $s_{t:j}$ and outputs a discrete action $a_t^j \in \{1, \ldots, K\}$, routing the j-th queued request r_j to the a_t^j-th LLM instance.

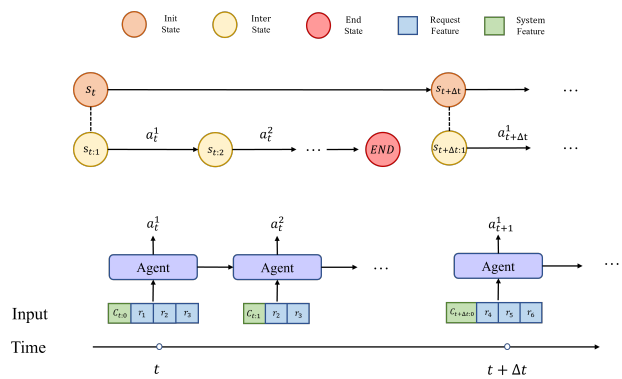


Figure 4: Markov Decision Process for global dispatch.

3.3.2 State Space. The state s_t encodes the current global system state, comprising features of the pending request set and the load status of each LLM instance.

Request state. The matrix $\mathbf{X}_t^{req}$ comprises four distinct feature vectors:

- $\vec{p}$ is the input length vector, where p_i denotes the number of input tokens for the i-th queued request.
- $\vec{d}$ is the estimated output length vector, where d_i represents the predicted output length of the i-th request.
- $\vec{u}$ is the parallel-stage indicator vector, where $u_i \in \{0, 1\}$ flags whether the i-th request resides in a parallelizable workflow stage.
- $\vec{\kappa}$ is the workflow type vector, where κ_i uniquely identifies the Agent application class of the i-th request.

Regarding the estimated output length d_i, we follow the methodology in [12] by predicting discrete length intervals rather than exact token counts. This coarse-grained approximation proves sufficient, as our designed MDP relies solely on relative interval magnitudes. Empirical evaluations demonstrate an interval prediction accuracy of approximately 70%.

Cluster state. The matrix $\mathbf{X}_t^{inst}$ is constructed by concatenating individual per-instance feature vectors. Specifically, the runtime profile of each LLM instance is characterized by:

- $\vec{m}$ and $\vec{g}$: One-hot encoded vectors specifying the instance's deployed model architecture and underlying GPU hardware, respectively.
- $\vec{wq}$: A load histogram of the waiting queue, derived by binning input lengths with a configurable step size *bin_width*.
- $\vec{rq}$: A load histogram of active execution queues. Prefill-stage requests are binned by input token length, whereas Decode-stage requests are binned by estimated output length.
- c: GPU memory utilization, computed as the proportion of allocated KV-cache blocks to the total block capacity.
- s: The expected prefix-cache reuse rate for the pending request, quantified as the fraction of reusable tokens.
- b: The estimated batch execution latency, maintained via a sliding-window average of recent iteration times.

To ensure training stability, all features are strictly normalized. The comprehensive MDP state representation is thus formulated as $s_t = [\mathbf{X}_t^{req} \,\|\, \mathbf{X}_t^{inst}]$, where $\|$ denotes the concatenation operator.

3.3.3 Reward Function. Using the end-to-end workflow latency directly as a reward signal would cause severe reward sparsity and credit-assignment problems due to the asynchronous execution model of LLM inference systems. Instead, we design a *composite immediate reward* that provides feedback after every dispatch decision:

$$r_t = \lambda_k \cdot r_k + \lambda_{imb} \cdot r_{imb} + \lambda_p \cdot r_p + \lambda_T \cdot T(x_{t,i}^{inst}), \tag{6}$$

where the terms are:

- r_k: *Prefix cache reuse reward*, representing the expected KV-cache reuse rate on the target instance, encouraging lower time-to-first-token.
- r_{imb}: *Load-imbalance penalty*, measuring the impact of the current dispatch on global load uniformity.
- r_p: *Queue penalty*, equal to the current waiting-queue length of the target instance, discouraging dispatch to overloaded nodes.
- $T(x_{t,i}^{inst})$: *Throughput estimate*, denoting the expected token throughput of the target instance in its current state.

To quantify load imbalance r_{imb}, the total load of instance i is estimated from its active request distribution ($\vec{rq}$) and waiting queue distribution ($\vec{wq}$):

$$L_i = \sum_{j=1}^{N_{bin}} (C_j \cdot N_{run,j}) + \sum_{j=1}^{N_{bin}} (C_j \cdot N_{pend,j}), \tag{7}$$

where C_j is the center length of bin j and $N_{\cdot,j}$ is the count of requests in that bin. The normalized imbalance score is:

$$S_{imb} = \frac{\sum_{i=1}^{K} |L_i - \bar{L}|}{\sum_{i=1}^{K} L_i + \epsilon}, \tag{8}$$

with $\bar{L} = \frac{1}{K} \sum_i L_i$ and ϵ a small constant for numerical stability. We set $r_{imb} = -S_{imb}$.

For the throughput estimate $T(x_{t,i}^{inst})$, we train a lightweight neural network surrogate offline on tens of thousands of real scheduling trajectories using MSE loss, achieving approximately 10% prediction error.

3.3.4 Policy Network Architecture. To capture complex temporal dependencies and heterogeneous instance features, we design a *Transformer-based scheduling network* that jointly encodes the current target request, the waiting-queue context, and the real-time state of each instance. Figure 5 shows the network topology.

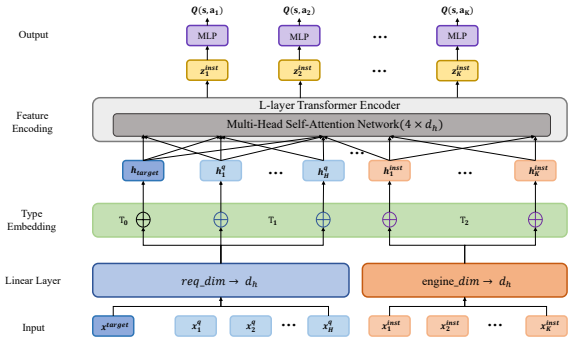


Figure 5: Dispatch policy network architecture.

The input layer is divided into three feature subspaces: the current target request $\mathbf{x}^{target}$, a look-ahead waiting queue $\{\mathbf{x}_i^q\}_{i=1}^H$ of length H, and the real-time states of K LLM instances $\{\mathbf{x}_j^{inst}\}_{j=1}^K$. Each feature vector is first projected to a unified hidden dimension d_h via a linear layer, then augmented with learned type embeddings to distinguish target, queue, and instance tokens (type $\in \{0, 1, 2\}$). The resulting sequence is processed by L Transformer encoder layers to capture cross-token dependencies. At the output layer, we extract the K instance-token representations $\{\mathbf{z}_{inst,j}\}_{j=1}^K$ and map each to an action-value score via a shared-parameter Q-value head:

$$Q(s, a_j) = \mathbf{W}_Q \cdot \mathrm{ReLU}\big(\mathbf{W}_H \mathbf{z}_{inst,j} + \mathbf{b}_H\big) + b_Q. \tag{9}$$

The final output $\mathbf{Q} \in \mathbb{R}^K$ gives the expected cumulative return for dispatching the current request to each instance.

3.3.5 Reinforcement Learning Training. We adopt a two-phase scheme combining offline pretraining and online fine-tuning.

Offline Pretraining. We construct a data-driven offline simulation environment by replaying real production workload traces covering diverse Agent workflows (e.g., multi-turn dialogue, code generation, document summarization) at various load intensities, improving generalization.

For the high-dimensional discrete action space, we adopt the *Double-DQN* algorithm [23], which decouples target action selection from value estimation to suppress Q-value overestimation. We replace the original MSE loss with Huber Loss [11], which applies linear penalty for large TD errors, significantly improving training stability under heterogeneous request sequences.

Online Fine-tuning. Although offline training establishes baseline scheduling capabilities, static policies often suffer performance degradation in highly dynamic production clusters. Consequently, we introduce an online fine-tuning mechanism. The agent continuously records interaction transitions into an experience replay buffer, triggering periodic parameter updates at a low learning rate once a predefined sample threshold is reached. To maintain Quality of Service stability, we bound the ϵ-greedy exploration rate and employ momentum-based soft updates for the target network, effectively mitigating Q-value oscillations under distribution shifts.

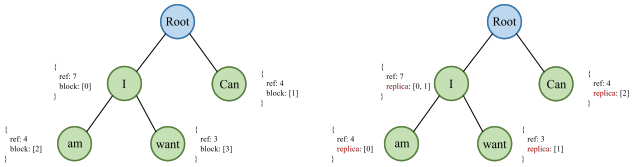


Figure 6: KV cache prefix trees: local (left) and global (right).

4 Implementation

To support RL agent training and system evaluation, we build our testbed upon Vidur [1], an open-source LLM inference simulator. Vidur natively supports heterogeneous GPUs (e.g., A100, H100) and provides built-in benchmarks for mainstream models like Llama and Qwen, enabling rapid validation of scheduling algorithms across diverse configurations.

However, native Vidur primarily targets single-turn LLM requests. It lacks the capability to simulate the multi-step interaction dependencies inherent in Agent workflows, nor does it support prefix-cache reuse. To address these limitations, we augment Vidur's core engine along two dimensions:

4.1 KV Cache Reuse

To enable KV cache reuse within the LLM engine (i.e., the local scheduler), we implement a `tree_cache` module atop Vidur's native `memory_pool`. This module comprises two core components: a *Block Manager* responsible for the allocation and reclamation of memory blocks, and a *Local Prefix Cache Tree* that maintains the prefix-mapping relationships of KV caches within each LLM instance.

Unlike sglang's single-level prefix tree, our design maintains both a per-instance local tree and a cluster-wide global tree augmented with replica mappings to enable cross-instance KV cache reuse. As shown in Figure 6, different request prefixes branch from the root node. Each node stores the token sequence, a reference count (`ref`) for memory reclamation, and a block index (`block`) mapping to the KV store.

To integrate with PagedAttention, only block-aligned prefixes are treated as valid cache entries, ensuring consistency with the underlying paged memory manager.

The *global KV cache prefix tree*, maintained by the global scheduler, records cache states across all LLM instances. Its structure mirrors the local tree, with the sole difference that each node stores a mapping from tree nodes to LLM instances (`replica`). When a user request arrives, the global scheduler can quickly locate matching instances through this tree, enabling cross-instance KV cache reuse.

4.2 Agent Workflow Support

Since Vidur cannot directly simulate complex Agent workflow logic, we extend it with a `UnifiedRequest` class that inherits from `Request` and adds `agent_type` and `agent_id` metadata for workflow identification.

`UnifiedRequest` further encodes a *workflow graph* of call-chain dependencies: subsequent stage requests are activated only after all predecessor requests in the graph have completed, faithfully simulating serial and parallel multi-step Agent interactions.

5 Evaluation

5.1 Experimental Setup

5.1.1 Hardware and Software Environment. All model training and experimental evaluation are conducted on a physical server running Ubuntu 18.04.6 LTS, equipped with dual Intel Xeon E5-2680 v3 processors (2.50 GHz) and two NVIDIA GeForce RTX 3090 GPUs.

For inference simulation we use the Vidur simulator. We emulate a four-node LLM inference cluster in which each node is equipped with a 40 GB NVIDIA A100 GPU and independently deploys one Meta-Llama-3-8B instance. The prefill chunk size (`chunk_size`) is set to 4096 and the maximum batch capacity (`batch_size`) to 512.

Agent workflows are implemented with the LangChain framework [5] to generate realistic request sequences and workflow topologies for the experiments.

5.1.2 RL Training Configuration. We set the learning rates for offline pretraining and online fine-tuning to 5×10^{-5} and 1×10^{-5}, respectively, with a discount factor $\gamma = 0.995$. The experience replay buffer is configured with a maximum capacity of 50,000 samples and a warm-up period of 500 samples. During training, the policy network, which features a hidden dimension of 512, is optimized using mini-batches of 256 samples. To ensure stability, the target network is updated every 100 steps, while the ε-greedy exploration rate is linearly decayed from 0.5 to 0.01.

For DynA, we set the look-ahead window $\Delta = 1$, the anti-starvation threshold $\tau_{\max} = 10.0$ s, and $\eta = 0.1$.

5.1.3 Agent Workloads. To comprehensively evaluate the proposed method across different Agent workflow types, we select three representative LLM Agent workloads covering serial, parallel, and hybrid topologies. Figure 7 shows the token-length and request-count distributions of each workload.

ChatBot (serial). We build approximately 8 000 multi-turn dialogue workflows from the ShareGPT dataset [8], each containing 2–20 dialogue rounds. The input length follows a right-skewed distribution (mean ≈837.54 tokens, concentrated in 0–2000 tokens), while output lengths are more concentrated (mean ≈191.78 tokens), reflecting the long-input/short-output pattern of conversation tasks.

Map-Reduce Summarization (parallel). We construct a Map-Reduce document summarization Agent: the Map phase splits long documents into chunks and issues concurrent LLM requests for sub-summaries; the Reduce phase aggregates them into a final output. Using the Wikidata dataset [24], we build approximately 1 000 workflows with 2–50 LLM requests each. The mean input length is ≈833.51 tokens with higher variance than ChatBot, and the mean output length is ≈186.51 tokens.

Code Generation Agent (hybrid). We design a multi-stage Agent following the pipeline User → Planner → Workers → Evaluators → Verifier → Final Output. If the code fails evaluation, a feedback loop triggers the Planner to re-schedule Workers for iterative repair. Using HumanEval [7] and MBPP [4], we build ≈1 000 workflows with 2–50 LLM requests each. The mean input length is ≈3541.76 tokens (significantly higher than the other two workloads, due to the feedback repair context), and the mean output length is ≈800.87 tokens.

Evaluation proceeds in two steps: first, each of the three workloads is tested in isolation; then a mixed workload combining all three types in preset ratios is evaluated. Workflow inter-arrival times follow a Poisson distribution. All experiments use the Llama3-8B model by default.

5.2 Baselines

We compare RAGS against the following baselines.

Round-Robin. Serving as our naive baseline, this strategy pairs First-Come-First-Serve (FCFS) queuing with round-robin dispatch. It treats all inference requests independently, operating entirely without Agent workflow awareness.

Parrot [14]. A state-of-the-art LLM serving system that uses semantic variable abstraction to expose inter-request DAG dependencies to the serving layer. It employs session affinity routing to maximize KV-cache prefix reuse while aggregating similar requests to satisfy application-level performance goals. We faithfully replicate its FCFS queuing and session affinity routing algorithms in our evaluation.

Ayo [22]. A fine-grained LLM orchestration framework that prioritizes requests based on their topological depth in the workflow DAG, with round-robin dispatch. We isolate and implement only its topology-aware queuing and routing logic for comparison.

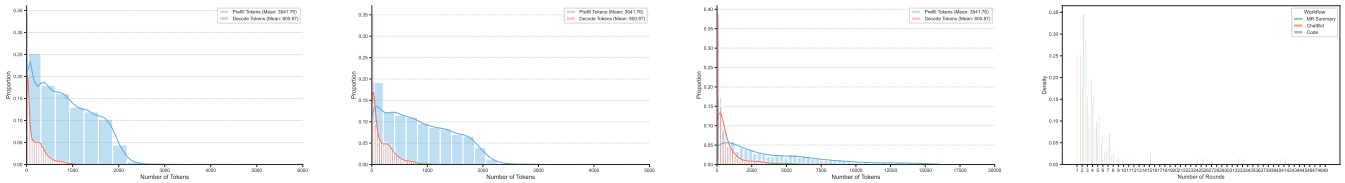
5.3 Main Results

Figure 8 shows the end-to-end performance of each method under the three single-type workloads at varying request arrival rates.

ChatBot (serial). At an arrival rate of 20 workflows/s, RAGS achieves up to **5.2**× lower mean E2E latency than Round-Robin and Ayo. For TTFT, Round-Robin's P50 surges to 17 s under high concurrency, while RAGS keeps it below 1 s through RL-based dynamic load balancing and DynA's priority queuing. RAGS's speedup over Parrot is only ≈1.1× in this scenario because ChatBot has a fixed topology with highly regular KV-cache sharing, allowing Parrot's static prefix-aware routing to effectively exploit the cache; the headroom for dynamic routing is therefore limited.

Map-Reduce Summarization (parallel). Since this workflow has a simple two-level topology, the overall gains of RAGS over baselines are moderate. At 20 workflows/s, RAGS reduces mean E2E latency by only ≈6% over Round-Robin. Ayo's topology-based priority essentially degenerates to round-robin because it cannot distinguish among the many same-level Map nodes. DynA, however, estimates workflow progress and preferentially schedules requests about to trigger the Reduce aggregation, partially alleviating inter-node synchronization blocking.

Code Generation Agent (hybrid). This workload introduces autonomous feedback-repair loops and is designed to assess the scheduler's ability to handle highly dynamic topologies. Parrot causes severe queue congestion in this scenario: its mean E2E latency reaches 240 s and P95 tail latency 600 s, approximately 3–4× higher than RAGS. The root cause is that the repair feedback loops create highly dynamic per-session compute loads, and static session affinity triggers severe global load imbalance, causing head-of-line



(a) Token Distribution of ShareGPT Dataset **(b) Token Distribution of Wikidata Dataset** **(c) Token Distribution of CodeAgent Dataset** **(d) Request Count Distribution of Different Workflows**

Figure 7: Feature Distributions of Different Agent Workflows

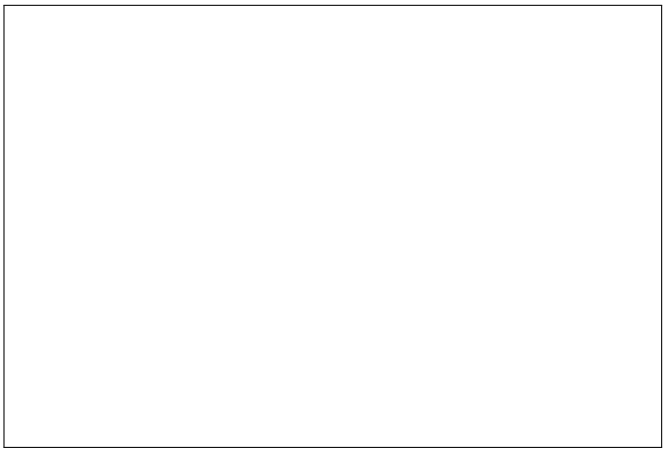


Figure 8: End-to-end performance comparison under three single-type workloads at varying request rates.

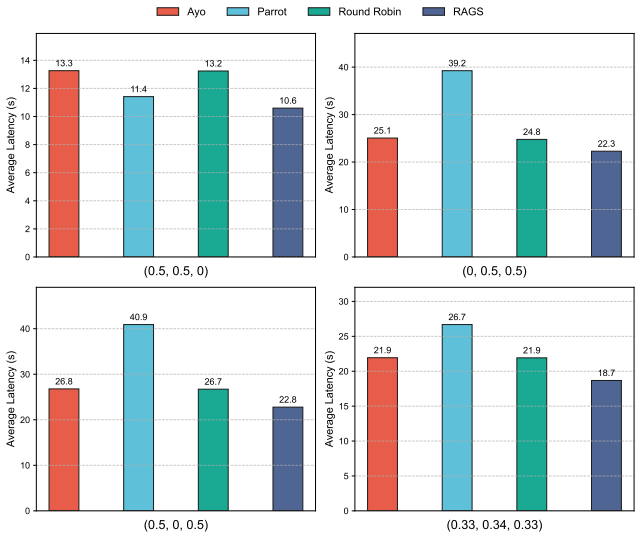


Figure 9: End-to-end performance comparison under mixed loads.

blocking on overloaded instances. Thanks to the synergy between RL-based dynamic load balancing and DynA's preemptive scheduling of high-priority requests, RAGS stabilizes mean latency in the range of 70–85 s across most load levels, and maintains P95 tail latency around 200 s even at 5 workflows/s.

TTFT further reveals the degree of queue congestion under highly dynamic load. Parrot's P50 TTFT spikes to ≈6 s at 5 workflows/s. RAGS, by jointly optimizing KV-cache reuse and load balance, keeps P50 TTFT below 0.5 s under the same heavy load, demonstrating that two-tier co-design effectively suppresses TTFT spikes in dynamic workflows.

Mixed workload. Real production environments feature highly heterogeneous mixed workloads. We construct a mixed dataset of 500 workflows and vary the proportions (x, y, z) of ChatBot, Summarization, and Code Agent workflows. The arrival rate is fixed at 5 workflows/s. Figure 9 presents mean E2E latency under four representative mixture ratios.

When the mix contains no Code Agent tasks (ratio $(0.5, 0.5, 0)$), most baselines converge to 11–14 s, with Parrot achieving 11.4 s (second best) and RAGS achieving 10.6 s (best). Once Code Agent tasks with complex feedback loops are introduced (e.g., ratios $(0, 0.5, 0.5)$ and $(0.5, 0, 0.5)$), Parrot's latency spikes to ≈40 s, exposing the inability of static binding to handle high compute density. Ayo and Round-Robin show similar degradation, confirming that static topology-depth sorting cannot cope with heterogeneous concurrent traffic. RAGS achieves 18.7 s under the balanced mix $(0.33, 0.34, 0.33)$, which is 14.5% lower than Ayo and 28.6% lower than Parrot, and keeps mean E2E latency below 22.8 s in every Code-Agent-inclusive scenario.

5.4 Hyperparameter Sensitivity

5.4.1 Effect of State-Space Bin Width. Figure 10 illustrates the impact of the load histogram's bin width on the mean E2E latency. An excessively narrow bin width results in sparse state representations, which hinders the convergence of the RL algorithm. Conversely, an overly broad bin width obscures fine-grained load variations across different instances. Empirical results indicate that a bin width of 64 strikes an optimal trade-off between representation fidelity and learning efficiency; thus, it is adopted as the default configuration.

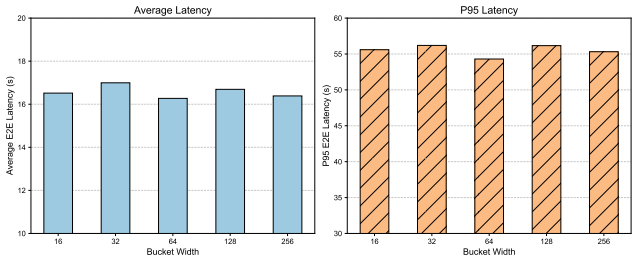


Figure 10: Effect of bin width on mean E2E latency.

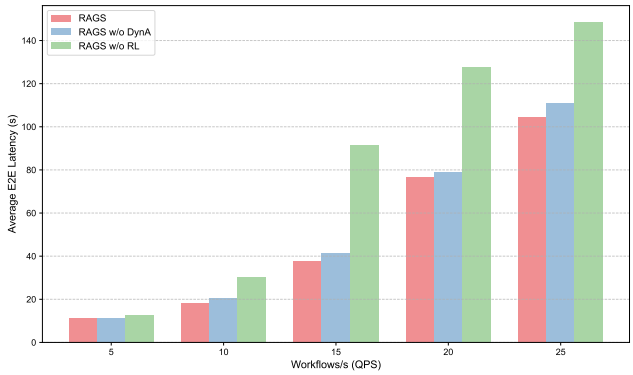


Figure 11: Component ablation under mixed workload.

5.5 Ablation Studies

To quantify the individual contributions of the queuing and dispatch components, we run an ablation on the mixed workload (ChatBot:Summarization:Code = 0.33:0.34:0.33) at varying arrival rates, comparing three configurations: (1) **RAGS**: full system (DynA + RL dispatch); (2) **RAGS w/o DynA**: FCFS queuing + RL dispatch; (3) **RAGS w/o RL**: DynA queuing + Round-Robin dispatch.

As shown in Figure 11, performance differences are small at low arrival rates (RAGS: 11.206 s, RAGS w/o DynA: 11.205 s, RAGS w/o RL: 12.867 s) but widen significantly at high loads. At the peak arrival rate, RAGS achieves 104.557 s; removing DynA raises latency to 110.804 s; removing RL dispatch degrades latency further to 148.758 s.

These results lead to three conclusions. First, RL dispatch is the dominant contributor: removing it increases peak mean latency by ≈42.27%, confirming that its benefit scales with system load. Second, DynA also provides non-trivial gains: at a medium arrival rate RAGS w/o DynA's latency is ≈14.8% higher than RAGS (20.782 s vs. 18.104 s), showing that workflow-progress-aware priority ordering effectively reduces head-of-line blocking. Third, the two components are complementary: DynA ensures that critical requests enter the dispatch stage first, while RL global load balancing prevents those requests from encountering secondary queuing inside LLM engines.

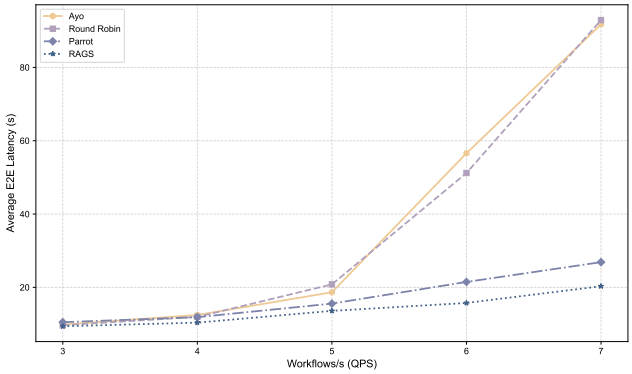


Figure 12: Heterogeneous GPU cluster evaluation results.

5.6 Heterogeneous Cluster Evaluation

To validate RAGS on a heterogeneous GPU cluster, we run experiments on a 4-GPU cluster with 2 NVIDIA A100s and 2 NVIDIA H100s. Since complete performance benchmarks for Meta-Llama-3-8B on H100 are not yet available, all instances deploy meta-llama/Llama-2-7b-hf. A100 and H100 differ significantly in compute throughput and memory bandwidth, so the scheduler must sense hardware heterogeneity while avoiding resource waste from unbalanced allocation.

As shown in Figure 12, all methods exhibit increasing mean E2E latency as the arrival rate rises from 3 to 7 workflows/s, but with markedly different slopes. Round-Robin ignores GPU heterogeneity and distributes evenly, turning the weaker A100 instances into system bottlenecks; its latency surges above 80 s at high concurrency. Parrot's static session-binding also lacks hardware-awareness: routing computationally intensive requests to weaker A100 instances easily triggers severe local overload.

RAGS maintains the lowest mean E2E latency at all load levels. Its advantage stems from the joint encoding of GPU type and instance load in the RL state space, which allows the policy to implicitly sense inter-instance compute differences and adaptively balance hardware performance with global load distribution. At the peak arrival rate of 7 workflows/s, RAGS reduces mean E2E latency by approximately 30%–40% compared with Round-Robin and Parrot, validating its effectiveness and generalizability in heterogeneous inference clusters.

6 Related Work

LLM Inference Engines. vLLM [13] introduces PagedAttention to manage KV cache with OS-style virtual memory paging, greatly reducing memory fragmentation and improving GPU utilization. SGLang [26] further proposes RadixAttention, indexing all historical KV caches in a radix tree to enable fine-grained cross-request prefix reuse and cache-aware routing. Sarathi-Serve [2] mitigates prefill–decode interference via chunked prefills, while Orca [25] pioneered continuous batching to keep GPUs fully occupied. These systems optimize single-turn request serving but lack mechanisms to model multi-step workflow dependencies inherent in agent workloads.

LLM Agent Serving Systems. Parrot [14] is among the first to address agent workflow scheduling: it introduces *semantic variables* to expose inter-request data-flow to the serving layer and adopts session-affinity routing to maximize KV cache reuse across correlated requests. Ayo [22] decomposes complex LLM applications into primitive-level dataflow graphs and co-schedules LLM and non-LLM stages in a pipeline, cutting end-to-end latency. Autellix [15] elevates the full agent program to the scheduling unit and proposes PLAS/ATLAS—multi-level feedback queue schedulers that use accumulated execution time on the critical path to mitigate head-of-line blocking. KVFlow [18] focuses on cache management for multi-agent workflows: it builds an agent execution graph, assigns priority to cache nodes based on estimated reuse distance, and employs asynchronous prefetching together with a future-reuse-aware eviction policy to raise cache hit rates. HexGen-Flow [20] proposes a two-level SLO-aware scheduler for Text-to-SQL pipelines, decomposing end-to-end budgets proportionally and using urgency-based preemption; however, it is tightly coupled to a specific pipeline structure and does not generalize to arbitrary agent topologies.

RL for Cluster Scheduling. DeepRM [16] and DL2 [19] demonstrate the effectiveness of deep RL for resource scheduling in DNN training clusters. TORTA [10] applies RL to LLM inference traffic routing across regions, dynamically adjusting per-window routing ratios. Unlike these works, RAGS applies RL to *agent workflow* routing, explicitly encoding workflow-level dependency states and KV cache residency to drive adaptive global dispatching.

References

[1] Amey Agrawal, Nitin Kedia, Jayashree Mohan, Ashish Panwar, Nipun Kwatra, Bhargav S Gulavani, Ramachandran Ramjee, and Alexey Tumanov. 2024. Vidur: A large-scale simulation framework for llm inference. *Proceedings of Machine Learning and Systems* 6 (2024), 351–366.

[2] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming {Throughput-Latency} tradeoff in {LLM} inference with {Sarathi-Serve}. In *18th USENIX symposium on operating systems design and implementation (OSDI)*, Ada Gavrilovska and Douglas B. Terry (Eds.). USENIX Association, Santa Clara, CA, 117–134.

[3] Amazon Web Services. 2026. *aws-genai-llm-chatbot: A modular and comprehensive solution to deploy a Multi-LLM and Multi-RAG powered chatbot.* https://github.com/aws-samples/aws-genai-llm-chatbot

[4] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732* (2021), 1–34.

[5] Harrison Chase. 2022. LangChain. https://github.com/langchain-ai/langchain. Accessed: 2026-03-13.

[6] Jinyuan Chen, Jiuchen Shi, Quan Chen, and Minyi Guo. 2025. Kairos: Low-latency multi-agent serving with shared llms and excessive loads in the public cloud. *arXiv preprint arXiv:2508.06948* (2025), 1–15.

[7] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021), 1–35.

[8] Wei-Lin Chiang, Zhuohan Li, Ziqing Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E Gonzalez, et al. 2023. *Vicuna: An Open-Source Chatbot Impressing GPT-4 with 90%* ChatGPT Quality*.* https://lmsys.org/blog/2023-03-30-vicuna/

[9] cognitivetech. 2026. *ollama-ebook-summary: LLM for Long Text Summary (Comprehensive Bulleted Notes).* https://github.com/cognitivetech/ollama-ebook-summary

[10] Chengze Du, Zhiwei Yu, Heng Xu, Haojie Wang, Jialong Li, et al. 2025. Temporal-aware gpu resource allocation for distributed llm inference via reinforcement learning. *arXiv preprint arXiv:2507.10259* (2025), 1–16.

[11] Peter J. Huber. 1992. Robust Estimation of a Location Parameter. In *Breakthroughs in Statistics: Methodology and Distribution*, Samuel Kotz and Norman L. Johnson (Eds.). Springer, New York, 492–518.

[12] Yunho Jin, Chun-Feng Wu, David Brooks, and Gu-Yeon Wei. 2023. S^3: Increasing GPU Utilization during Generative Inference for Higher Throughput. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, December 10–16, 2023, New Orleans, LA, USA (NeurIPS)*, Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine (Eds.). Curran Associates, Inc., Red Hook, NY, USA, 18015–18027.

[13] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles (SOSP)*, Jason Flinn and Margo I. Seltzer (Eds.). Association for Computing Machinery, Koblenz, Germany, 611–626.

[14] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. 2024. Parrot: Efficient serving of {LLM-based} applications with semantic variable. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, Ada Gavrilovska and Douglas B. Terry (Eds.). USENIX Association, Santa Clara, CA, 929–945.

[15] Michael Luo, Xiaoxiang Shi, Colin Cai, Tianjun Zhang, Justin Wong, Yichuan Wang, Chi Wang, Yanping Huang, Zhifeng Chen, Joseph E Gonzalez, et al. 2025. Autellix: An efficient serving engine for llm agents as general programs. *arXiv preprint arXiv:2502.13965* (2025), 1–17.

[16] Hongzi Mao, Mohammad Alizadeh, Ishai Menache, and Srikanth Kandula. 2016. Resource management with deep reinforcement learning. In *Proceedings of the 15th ACM workshop on hot topics in networks (HotNets)*, Bryan Ford and Alex C. Snoeren (Eds.). Association for Computing Machinery, Atlanta, GA, 50–56.

[17] OpenCode AI. 2026. *OpenCode: A powerful AI coding agent. Built for the terminal.* https://github.com/opencode-ai/opencode

[18] Zaifeng Pan, Ajjkumar Patel, Zhengding Hu, Yipeng Shen, Yue Guan, Wan-Lu Li, Lianhui Qin, Yida Wang, and Yufei Ding. 2025. KVFlow: Efficient prefix caching for accelerating LLM-based multi-agent workflows. *arXiv preprint arXiv:2507.07400* (2025), 1–12.

[19] Yanghua Peng, Yixin Bao, Yangrui Chen, Chuan Wu, Chen Meng, and Wei Lin. 2021. DL2: A deep learning-driven scheduler for deep learning clusters. *IEEE Transactions on Parallel and Distributed Systems* 32, 8 (2021), 1947–1960.

[20] You Peng, Youhe Jiang, Wenqi Jiang, Chen Wang, and Binhang Yuan. 2025. HEXGEN-FLOW: Optimizing LLM Inference Request Scheduling for Agentic Text-to-SQL. *arXiv preprint arXiv:2505.05286* (2025), 1–21.

[21] Vikranth Srivatsa, Zijian He, Reyna Abhyankar, Dongming Li, and Yiying Zhang. 2024. Preble: Efficient distributed prompt scheduling for llm serving. *arXiv preprint arXiv:2407.00023* (2024), 1–23.

[22] Xin Tan, Yimin Jiang, Yitao Yang, and Hong Xu. 2025. Towards end-to-end optimization of llm-based applications with ayo. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS)*, Adrian Sampson, Martha Kim, and Christopher J. Rossbach (Eds.). Association for Computing Machinery, Rotterdam, Netherlands, 1302–1316.

[23] Hado Van Hasselt, Arthur Guez, and David Silver. 2016. Deep reinforcement learning with double q-learning. In *Proceedings of the AAAI conference on artificial intelligence (AAAI)*, Dale Schuurmans and Michael P. Wellman (Eds.), Vol. 30. AAAI Press, Phoenix, AZ, 2094–2100.

[24] Denny Vrandečić and Markus Krötzsch. 2014. Wikidata: A Free Collaborative Knowledgebase. *Commun. ACM* 57, 10 (2014), 78–85.

[25] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A distributed serving system for {Transformer-Based} generative models. In *16th USENIX symposium on operating systems design and implementation (OSDI)*, Marcos K. Aguilera and Hakim Weatherspoon (Eds.). USENIX Association, Carlsbad, CA, 521–538.

[26] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark W. Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, December 10–15, 2024, Vancouver, BC, Canada (NeurIPS)*, Amir Globerson, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang (Eds.). Curran Associates, Inc., Red Hook, NY, USA, 62557–62583.